Python Engineering - Reference Guide

Class 08 - Data Processing con NumPy - Version ampliada

Guia de referencia para mascaras booleanas, filtrado, estadistica y procesado de datos. Esta version anade, para cada funcion, una explicacion breve de que hace, que recibe, que devuelve y un ejemplo de uso.

1. El patron mental del procesado de datos

```
ENGINEERING QUESTION
        -> CONDITION
           -> BOOLEAN MASK
              -> FILTER / COUNT / INDICES
                 -> STATISTIC / RESULT
```

2. Que es una mascara booleana?

Una mascara booleana es un array de valores True/False. Cada posicion de la mascara corresponde a una posicion del array original.

```
heights = np.array([20, 80, 120, 150])

mask = heights > 100

# mask -> [False False True True]
```

3. Que significa A[mask]?

`A[mask]` no es una funcion: es **indexacion booleana**. Selecciona de A solo los elementos o filas cuya posicion correspondiente en mask sea True.

```
A = np.array([
    [10, 1],
    [20, 2],
    [30, 3]
])

mask = np.array([True, False, True])

selected = A[mask]

# [[10, 1],
#  [30, 3]]
```

Si A es 2D y mask tiene una entrada por fila, NumPy usa la mascara para seleccionar filas completas.

4. Operadores para combinar condiciones

Operacion	Sintaxis NumPy	Ejemplo	Devuelve
AND	&	(h > 50) & (v > 0)	True donde ambas condiciones son True
OR	|	(h > 100) | (v < -40)	True donde al menos una condicion es True
NOT	~	~(v > 0)	invierte True/False

Con arrays NumPy, cada condicion debe ir normalmente entre parentesis. No usamos and/or/not para comparaciones elemento a elemento.

5. np.sum(mask)

Que hace: suma los elementos. En una mascara booleana, True se comporta como 1 y False como 0.

Que recibe: un array o expresion NumPy; en este caso, una mascara booleana.

Que devuelve: un numero con la suma. Si la entrada es una mascara, equivale al numero de True.

```
mask = np.array([True, False, True, True])

count = np.sum(mask)
# 3
```

6. np.where()

Que hace: busca las posiciones donde se cumple una condicion.

Que recibe: normalmente una condicion o mascara booleana.

Que devuelve: una tupla de arrays de indices, uno por cada dimension. Por eso, para un array 1D, solemos escribir `np.where(mask)[0]`.

```
heights = np.array([20, 80, 120, 150])
mask = heights > 100

indices = np.where(mask)[0]

# [2 3]
```

`heights[mask]` devuelve los valores seleccionados; `np.where(mask)[0]` devuelve sus posiciones.

7. np.mean()

Que hace: calcula la media aritmetica de los valores.

Que recibe: un array numerico; opcionalmente puedes indicar un axis.

Que devuelve: un numero o un array, dependiendo de si trabajas sobre todo el array o por eje.

```
x = np.array([10.0, 20.0, 30.0])
mean_x = np.mean(x)
# 20.0
```

8. np.std()

Que hace: calcula la desviacion estandar, una medida de dispersion alrededor de la media.

Que recibe: un array numerico; opcionalmente un axis.

Que devuelve: un numero o array con la desviacion estandar.

```
x = np.array([10.0, 20.0, 30.0])
std_x = np.std(x)
```

9. np.min() y np.max()

Que hacen: buscan el valor minimo o maximo.

Que reciben: un array numerico; opcionalmente un axis.

Que devuelven: el minimo/maximo global o uno por eje.

```
np.min(heights)
np.max(heights)

np.max(matrix, axis=0)
# maximum per column
```

10. np.median()

Que hace: calcula la mediana, el valor central tras ordenar los datos.

Que recibe: un array numerico.

Que devuelve: un numero o array si se usa axis.

```
x = np.array([1, 2, 100])
np.median(x)
# 2.0
```

11. np.diff()

Que hace: calcula diferencias consecutivas: x[i+1] - x[i].

Que recibe: un array numerico ordenado.

Que devuelve: un array con un elemento menos que el original.

```
times = np.array([0.0, 0.1, 0.2, 0.3])
dt = np.diff(times)

# [0.1 0.1 0.1]
```

12. np.column_stack()

Que hace: coloca varios vectores 1D como columnas de una matriz 2D.

Que recibe: una tupla/lista de arrays que tengan la misma longitud.

Que devuelve: una matriz 2D con una columna por cada array de entrada.

```
processed = np.column_stack((
    heights,
    velocities,
    times,
    energy
))

# shape -> (N, 4)
```

13. np.allclose()

Que hace: comprueba si dos arrays o valores son aproximadamente iguales dentro de una tolerancia.

Que recibe: dos valores/arrays comparables; opcionalmente tolerancias rtol y atol.

Que devuelve: un unico booleano: True si todos los elementos son suficientemente cercanos; False si no.

```
dt_samples = np.diff(times)

is_constant = np.allclose(
    dt_samples,
    config['dt']
)

# True or False
```

14. Filtrar por rango

```
mask = (
    (times >= 2.0)
    & (times <= 5.0)
)

window = trajectory_array[mask]
```

Entrada: condiciones booleanas sobre el array. Salida: una mascara que luego puede usarse para obtener solo las filas dentro del rango.

15. Resumen ampliado

Sintaxis	Que recibe	Que devuelve	Para que sirve
x[mask]	array + mask booleana	valores seleccionados	filtrar elementos
A[mask]	matriz + mask por filas	filas seleccionadas	filtrar estados
np.sum(mask)	mask booleana	numero	contar True
np.where(mask)[0]	mask booleana	indices	saber donde ocurre

Sintaxis	Que recibe	Que devuelve	Para que sirve
np.mean(x)	array numerico	media	valor promedio
np.std(x)	array numerico	desviacion	dispersion
np.diff(x)	array ordenado	N-1 diferencias	cambios consecutivos
np.column_stack(...)	vectores iguales en longitud	matriz 2D	anadir variables
np.allclose(a,b)	dos valores/arrays	True/False	comparar floats